NGÔN NGỮ PHP

Nguyễn Hoàng Tùng
Bộ môn Kỹ thuật phần mềm
nhoangtung@agu.edu.vn
www.nhtung.id.vn

Nội dung chính của phần này

- **Giới thiệu**
- Làm sao “chạy” được PHP?
- Cơ bản về PHP
- Biến "Superglobals"
- Form
- Mảng
- Hàm
- Một số hàm PHP thường dùng
- Session
- Hướng đối tượng trong PHP

Giới thiệu

- PHP là viết tắt của “**P**HP: **H**ypertext **P**reprocessor”.
- Ra đời năm 1994 bởi Rasmus Lerdorf (1968, Greenland) sau đó được phát triển bởi nhiều người trải qua nhiều phiên bản. Phiên bản hiện tại là PHP 8.
- PHP là một ngôn ngữ kịch bản (Scripting language).
- Một tập tin PHP có thể chứa văn bản, HTML, CSS, JS và mã PHP.
- Mã PHP được thực thi trên máy chủ và kết quả sẽ được trả về cho trình duyệt dưới dạng HTML.
- PHP là mã nguồn mở miễn phí. Tải về từ nguồn PHP chính thức: www.php.net.

Nội dung chính của phần này

- Giới thiệu
- **Làm sao “chạy” được PHP?**
- Cơ bản về PHP
- Biến "Superglobals"
- Form
- Mảng
- Hàm
- Một số hàm PHP thường dùng
- Session
- Hướng đối tượng trong PHP

Làm sao "chạy" được PHP?

- Để "chạy" được PHP ta phải:
  - Tìm một web host hỗ trợ PHP, hoặc
  - Tự cài một web server hỗ trợ PHP trên máy cá nhân.
- Có rất nhiều web server hỗ trợ PHP:
  - **Apache**, **Nginx**, **Lighttpd** trên Unix.
  - **MAMP** trên macOS
  - **Internet Information Services** (IIS) trên Windows.
  - ...
- Trong phạm vi học phần này sẽ sử dụng web server **Apache** làm công cụ demo.

Làm sao “chạy” được PHP?

- Có rất nhiều phần mềm tạo web server Apache.


Vertrigo


Wamp


XAMPP


AppServ


USBWebServ

- Khi cài đặt một trong các phần mềm đó, sẽ có các dịch vụ:
  - Apache HTTP server
  - PHP
  - MySQL
  - phpMyAdmin
  - …
- Lưu ý: Nếu xung đột cổng **80** thì phải đổi sang cổng khác.

Nội dung chính của phần này

- Giới thiệu
- Làm sao “chạy” được PHP?
- **Cơ bản về PHP**
- Biến "Superglobals"
- Form
- Mảng
- Hàm
- Một số hàm PHP thường dùng
- Session
- Hướng đối tượng trong PHP

Cơ bản về PHP

- Cú pháp cơ bản:
  - Mã PHP có thể chèn vào bất cứ nơi nào trong file HTML.
  - Chèn mã PHP vào file HTML:

```
<?php echo "Hello World!"; ?>
```

  - Phần mở rộng của tập tin chứa mã PHP thường là **.php**

```
<!DOCTYPE html>
<html>
    <head>
        <title>Hello</title>
        <meta charset="utf-8" />
    </head>
    <body>
        <h3>Trang PHP đầu tiên</h3>
        <?php echo "Hello World!"; ?>
    </body>
</html>
```

Cơ bản về PHP

- Cú pháp cơ bản:
  - Phân biệt chữ hoa và chữ thường.
  - Chú thích trong PHP:

```
// Chú thích trên một dòng

#  Chú thích trên một dòng

/*
   Chú thích trên
   nhiều dòng
*/
```

Cơ bản về PHP

- Biến, hằng, kiểu dữ liệu:
  - Kiểu dữ liệu: PHP hỗ trợ **8** kiểu dữ liệu.
    - Integer, Float, Boolean, String, Array, Object, NULL, Resource.
  - Khai báo biến: Trong PHP, một biến bắt đầu bằng dấu **$**, theo sau là tên của biến.
    - $tên_biến = giá_trị;
    - $tên_mảng = **array**();
    - $tên_object = **new** tên_lớp();
  - Khai báo hằng:
    - **define**("tên_hằng", giá_trị);

```php
<?php
    define("PI", 3.14);
    $banKinh = 4;
    echo "Diện tích hình tròn: " . $banKinh * $banKinh * PI;
?>
```

Cơ bản về PHP

- Lưu ý:
  - Tên biến bắt đầu bằng dấu đô la ($),
  - Tên biến bao gồm chữ cái, chữ số, dấu gạch dưới (_),
  - Tên biến phải bắt đầu bằng chữ cái hoặc dấu gạch dưới (_),
  - Tên biến không chứa dấu và khoảng trắng,
  - Không sử dụng dấu $ trước tên hằng số,
  - Hằng số không có giới hạn phạm vi và không được định nghĩa lại.

Cơ bản về PHP

- Chuyển đổi kiểu dữ liệu: Việc ép kiểu trong PHP được thực hiện bằng cách:
  - `(string)` - Chuyển đổi sang kiểu String.
  - `(int)` - Chuyển đổi sang kiểu Integer.
  - `(float)` - Chuyển đổi sang kiểu Float.
  - `(bool)` - Chuyển đổi sang kiểu Boolean.
  - `(array)` - Chuyển đổi sang kiểu Mảng.
  - `(object)` - Chuyển đổi sang kiểu Object.

Cơ bản về PHP

Các phép toán:

- Phép gán: `=`, `+=`, `-=`, `*=`, `/=`, `%=`
- Phép toán số học: `+`, `-`, `*`, `/`, `%`, `**`
- Phép toán so sánh: `==`, `===`, `!=`, `!==`, `<>`, `>=`, `<=`, `>`, `<`, `<=>`
- Phép toán tăng/giảm: `++`, `--`
- Phép toán logic: `!`, `&&`, `and`, `||`, `or`
- Phép gán có điều kiện: `?:`, `??`
- Phép ghép chuỗi: `.` (dấu chấm), `.=`
- Phép toán xử lý mảng: `+`, `==`, `===`, `!=`, `!==`, `<>`

Cơ bản về PHP

Các câu lệnh điều khiển:

- Câu lệnh **IF**:
  - if (điều kiện) { /* nhóm lệnh cần thực thi */ }
  - if (điều kiện) { /* nhóm lệnh 1 */ } else { /* nhóm lệnh 2 */ }
  - if (điều kiện 1) { /* nhóm lệnh 1 */ }
    elseif (điều kiện 2) { /* nhóm lệnh 2 */ }
    …
    elseif (điều kiện n) { /* nhóm lệnh n */ }
    else { /* nhóm lệnh cuối cùng */ }

Cơ bản về PHP

Các câu lệnh điều khiển:

- Câu lệnh **SWITCH**:

```
switch ($biến) {
    case giá_trị_1: nhóm lệnh 1; break;
    case giá_trị_2: nhóm lệnh 2; break;
    …
    case giá_trị_n: nhóm lệnh n; break;
    default: nhóm lệnh n + 1;
}
```

Cơ bản về PHP

Các câu lệnh điều khiển:

- Câu lệnh **WHILE**:

```
while (điều kiện) {
    /* nhóm lệnh cần thực thi */
}
```

- Câu lệnh **DO...WHILE**:

```
do {
    /* nhóm lệnh cần thực thi */
} while (điều kiện);
```

Cơ bản về PHP

Các câu lệnh điều khiển:

- Câu lệnh **FOR**:

```
for ($khởi_tạo; điều kiện; biểu thức) {
    /* nhóm lệnh cần thực thi */
}
```

- Câu lệnh **FOREACH**:

```
foreach ($mảng as $key => $value) {
    /* nhóm lệnh cần thực thi */
}
```

Nội dung chính của phần này

- Giới thiệu
- Làm sao “chạy” được PHP?
- Cơ bản về PHP
- **Biến "Superglobals"**
- Form
- Mảng
- Hàm
- Một số hàm PHP thường dùng
- Session
- Hướng đối tượng trong PHP

Biến "Superglobals"

- Một số biến định nghĩa sẵn trong PHP gọi là "*Superglobals*", các biến này lúc nào cũng có thể truy cập được, phạm vi nào cũng có thể truy cập được.
- Một số biến "*Superglobals*" thường dùng:
  - $_SERVER
  - $_REQUEST, $_POST, $_GET
  - $_FILES
  - $_COOKIE
  - $_SESSION
- Xem thêm: *http://php.net/manual/en/language.variables.superglobals.php*

Biến "Superglobals"

- Ví dụ:

```php
<?php
    // Trả về tên file của script hiện hành
    echo $_SERVER['PHP_SELF'];

    // Trả về tên của server, ví dụ www.nhtung.com
    echo $_SERVER['SERVER_NAME'];

    // Trả về tên của server, ví dụ www.nhtung.com:8080
    echo $_SERVER['HTTP_HOST'];

    // Xem thêm: http://php.net/manual/en/reserved.variables.server.php
    echo $_SERVER['HTTP_REFERER'];
    echo $_SERVER['HTTP_USER_AGENT'];
    echo $_SERVER['SCRIPT_NAME'];
    echo $_SERVER['SERVER_ADDR'];
    echo $_SERVER['SERVER_PORT'];
?>
```

Nội dung chính của phần này

- Giới thiệu
- Làm sao “chạy” được PHP?
- Cơ bản về PHP
- Biến "Superglobals"
- **Form**
- Mảng
- Hàm
- Một số hàm PHP thường dùng
- Session
- Hướng đối tượng trong PHP

Form

- Các thuộc tính của FORM:
  - **action**: chỉ đến tập tin PHP sẽ nhận dữ liệu từ FORM.
  - **method**: có 2 dạng phương thức
    - **GET**: Dữ liệu truyền từ client đến server "công khai", các giá trị và thông tin nhập liệu từ các input sẽ hiện trên thanh địa chỉ.
    - **POST**: Các giá trị và thông tin nhập liệu từ các input sẽ **không** hiện trên thanh địa chỉ.
  - Cú pháp chung:

```
<form action="input.php" method="post" >
    <!– Danh sách các input của form -->
</form>
```

Form

- Lấy dữ liệu từ FORM:
  - Đối với GET:
    - `$hten  = $_GET['hoten'];`
    - `$gtinh  = $_GET['gioitinh'];`
  - Đối với POST:
    - `$uname = $_POST['username'];`
    - `$pword = $_POST['password'];`
  - Lưu ý:
    - `$_GET[], $_POST[]` là các biến siêu toàn cục.
    - Có thể thay thế `$_GET[], $_POST[]` bằng **`$_REQUEST[]`**.

Nội dung chính của phần này

- Giới thiệu
- Làm sao “chạy” được PHP?
- Cơ bản về PHP
- Biến "Superglobals"
- Form
- **Mảng**
- Hàm
- Một số hàm PHP thường dùng
- Session
- Hướng đối tượng trong PHP

Mảng

- Khai báo:
  - $arr1 = array();
  - $arr2 = [];
  - $arr3 = [giá trị 1, giá trị 2,..., giá trị n];
  - $arr4 = ['hoten' => 'Nguyễn Hoàng Tùng', // Mảng kết hợp
    'diachi' => 'Long Xuyên – An Giang',
    'namsinh' => 1896,
    'IQ' => 'Rất cao'];



Mảng

- Sử dụng:
  - Truy xuất 1 phần tử trong mảng: $arr[$key]

    → $key có ý nghĩa rộng hơn index, dùng tham chiếu đến 1 phần tử của mảng.
  - *Ví dụ:* Tạo và xuất mảng gồm 10 phần tử là các số nguyên lấy ngẫu nhiên trong [0,100].

```php
<?php
    $arr = [];

    for($i = 0; $i < 10; $i++)
        $arr[$i] = rand(0, 100);

    for($i = 0; $i < 10; $i++)
        echo($i + 1) . " - " . $arr[$i] . "<br />";
?>
```

  - Xóa 1 phần tử trong mảng: **unset**($arr[$key])

Mảng

- Duyệt mảng:
  - **foreach()**: truy xuất các phần tử của mảng

```
foreach($arr as $key => $value)
{
    echo $key . ", ";
    echo $value . "<br />";
}
```

→ Có thể thay thế **$key** và **$value** bằng tên khác, chẳng hạn **$k** => **$v**, miễn rằng đảm bảo khai báo: $chỉ số => $giá trị.

Mảng

- Các hàm về mảng:
  - **count()**, **sizeof()**: trả về kích thước của mảng.
  - **sort()**: sắp xếp mảng theo **value** của phần tử.
  - **ksort()**: sắp xếp mảng theo **key** của phần tử.
  - **is_array()**: kiểm tra 1 biến có phải là mảng (trả về 1 hoặc 0).
  - **shuffle()**: xáo trộn các phần tử trong mảng.
  - Xem thêm: http://php.net/manual/en/ref.array.php

Nội dung chính của phần này

- Giới thiệu
- Làm sao “chạy” được PHP?
- Cơ bản về PHP
- Biến "Superglobals"
- Form
- Mảng
- **Hàm**
- Một số hàm PHP thường dùng
- Session
- Hướng đối tượng trong PHP

Hàm

- Cú pháp:

```
function tên_hàm([các_tham_số])
{
    // Khai báo biến
    // Câu lệnh PHP
    [return giá_trị;]
}
```

- Sử dụng:
  - Hàm không có trị trả về:
    - `tên_hàm([các_tham_số]);`
  - Hàm có trị trả về:
    - Tham gia vào các biểu thức, gán giá trị cho biến.

Hàm có tham số mặc định

- Khi không có tham số, hàm sẽ sử dụng tham số mặc định.
- Tham số mặc định phải đặt phía cuối cùng trong danh sách các tham số.

```php
<?php
    function Hello($name = "Không có ai cả")
    {
        echo "Xin chào $name";
    }

    Hello();       // Xin chào Không có ai cả
    Hello("Tuấn");   // Xin chào Tuấn
?>
```

Mở rộng: Sử dụng lại file PHP đã có

- **require**(“đường dẫn đến tập tin”)
- **require_once**(“đường dẫn đến tập tin”)

- **include**(“đường dẫn đến tập tin”)
- **include_once**(“đường dẫn đến tập tin”)

Mở rộng: Sử dụng lại file PHP đã có

❑ require() & include()

- Giống: Chèn file vào file hiện tại, nếu file được chèn không tìm thấy thì hiện thông báo lỗi.
- Khác: Khi file được chèn bằng lệnh **require()** mà file đó không tìm thấy thì trình thông dịch sẽ dừng lại, không dịch nữa và sẽ xuất hiện thông báo lỗi. Còn khi file được chèn bằng lệnh **include()** mà file đó không tìm thấy thì trình biên dịch vẫn tiếp tục dịch cho đến hết, đồng thời cũng xuất hiện warning để mình biết file đó bị lỗi.

❑ require_once() & include_once()

- Là hai dạng biến đổi của hàm require() và include().
- Ý nghĩa: Nếu file được chèn đã được chèn ở trước đó rồi, thì bây giờ sẽ không chèn nữa.

Mở rộng: Sử dụng lại file PHP đã có

- Ví dụ:
  - header.php

```
<h3>Trang web của tôi!</h3>
<p>Xin chào các bạn đến với trang web này!</p>
```

  - footer.php

```
<?php
    echo "<p>Bản quyền &copy; " . date('Y') . " bởi NHTùng</p>";
?>
```

Mở rộng: Sử dụng lại file PHP đã có

- Ví dụ:
  - index.php

```
<!DOCTYPE html>
<html>
    <head>
        <title>Demo</title>
        <meta charset="utf-8" />
    </head>
    <body>
        <?php include_once "header.php"; ?>

        <p>Nội dung trang web.</p>

        <?php include_once "footer.php"; ?>
    </body>
</html>
```

Nội dung chính của slide này

- Giới thiệu
- Làm sao “chạy” được PHP?
- Cơ bản về PHP
- Biến “Superglobals”
- Form
- Mảng
- Hàm
- **Một số hàm PHP thường dùng**
- Session
- Hướng đối tượng trong PHP

Một số hàm PHP thường dùng

- Hàm kiểm tra
- Hàm về chuỗi
- Hàm bảo mật
- Hàm ngày, giờ
- Upload file lên server

Một số hàm PHP thường dùng

- Hàm kiểm tra:
  - **is_array**($var) – Kiểm tra biến $var có phải kiểu array không?
  - **is_null**($var) – Kiểm tra biến $var có giá trị null không?
  - **is_numeric**($var) – Kiểm tra biến $var có phải là số không?
  - **isset**($var) – Kiểm tra biến $var có tồn tại không?
  - **file_exists**($path) – Kiểm tra thư mục hoặc tập tin có tồn tại không?
  - **empty**($var) – Kiểm tra biến $var có rỗng không?
  - **in_array**($var, $array) – Kiểm tra biến $var có trong $array không?
  - **array_key_exists**($key, $array) – Kiểm tra chỉ số $key có trong $array không?
  - **checkdate**($month, $day, $year) – Kiểm tra ngày tháng năm có hợp lệ không?

Một số hàm PHP thường dùng

- Hàm kiểm tra:
  - bool **is_null**(mixed $var) – Kiểm tra biến $var có giá trị null không?

```php
<?php
    $a = "";
    $b = 0;
    $c = null;

    if(is_null($a)) echo "A - TRUE";

    if(is_null($b)) echo "B - TRUE";

    if(is_null($c)) echo "C - TRUE";

    if(is_null($d)) echo "D - TRUE";
?>
```

Một số hàm PHP thường dùng

- Hàm kiểm tra:
  - bool **isset**(mixed $var [, mixed $...]) – Kiểm tra biến $var,... có tồn tại không?

```php
<?php
    $a = "";
    $b = 0;
    $c = null;

    if(isset($a)) echo "A - TRUE";

    if(isset($b)) echo "B - TRUE";

    if(isset($c)) echo "C - TRUE";

    if(isset($d)) echo "D - TRUE";
?>
```

Một số hàm PHP thường dùng

- Hàm kiểm tra:
  - bool **empty**(mixed $var) – Kiểm tra biến $var có rỗng không?

```php
<?php
    $a = "";        $b = "0";       $c = 0;         $d = 0.0;
    $e = null;      $f = false;     $g = array();   $h;

    if(empty($a) && empty($b) && empty($c) && empty($d) &&
       empty($e) && empty($f) && empty($g) && empty($h))
    {
       echo "TRUE";
    }
?>
```

Một số hàm PHP thường dùng

- Hàm về chuỗi:
  - **explode**($delimiter, $str) – Tách chuỗi $str thành các chuỗi con.
  - **implode**($glue, $arr) – Ghép các phần tử của mảng $arr thành chuỗi.
  - **ltrim**($str), **rtrim**($str), **trim**($str) – Xóa khoảng trắng bên trái / phải / cả hai.
  - **number_format**($number, $decimals) – Định dạng số.
  - **str_replace**($search, $replace, $subject) – Tìm và thay thế chuỗi.
  - **strlen**($str) – Trả về chiều dài của chuỗi $str.
  - **strtolower**($str) – Chuyển chuỗi $str sang chữ thường.
  - **strtoupper**($str) – Chuyển chuỗi $str sang chữ HOA.
  - **ucfirst**($str) – Chuyển chuỗi $str sang dạng viết hoa đầu câu.
  - **ucwords**($str, $delimiter) – Chuyển chuỗi $str sang viết hoa đầu mỗi từ.

Một số hàm PHP thường dùng

- Hàm về chuỗi:
  - array **explode**(string $delimiter, string $str)
  - Tách chuỗi $str thành các chuỗi con dựa theo dấu phân cách $delimiter.

```php
<?php
    $str = "02/09/2017";
    $arr = explode("/", $str);

    echo "Ngày: " . $arr[0];
    echo "Tháng: " . $arr[1];
    echo "Năm: " . $arr[2];

    $str2 = "Xoài, Mận, Bưởi, Sầu riêng";
    $arr2 = explode(", ", $str2);

    echo $arr2[3];
?>
```

Một số hàm PHP thường dùng

- Hàm về chuỗi:
  - string **implode**(string $glue, array $arr)
  - Ghép các phần tử của mảng $arr thành chuỗi có ký tự kết nối $glue.

```php
<?php
    $arr = array("NV01", "NV03", "NV08", "NV10");
    $str = implode(",", $arr);

    $sql = "SELECT *
            FROM tbl_nhanvien
            WHERE MaNV IN($str)";
    echo $sql;
?>
```

Một số hàm PHP thường dùng

❑ Hàm bảo mật:

- **addslashes**($str) – Thêm dấu **** trước các ký tự ' " \ NUL trong chuỗi $str.
- **stripslashes**($str) – Xóa bỏ dấu **** xuất hiện trong chuỗi $str.
- **htmlspecialchars**($str) – Chuyển các ký tự đặc biệt trong $str thành HTML.
- **htmlspecialchars_decode**($str) – Ngược lại với hàm **htmlspecialchars**.
- **md5**($str) – Tạo ra chuỗi mã băm có độ dài 128 bit từ chuỗi đầu vào $str.
- **sha1**($str) – Tạo ra chuỗi mã băm có độ dài 160 bit từ chuỗi đầu vào $str.
- **password_hash**($str) – Băm mật khẩu theo thuật toán có sử dụng muối (salt).
- **password_verify**($str, $hash) – Kiểm tra mật khẩu đã băm bằng hàm **password_hash**.

Một số hàm PHP thường dùng

string **addslashes**(string $str)

- **Thêm** ký tự backslash (\) phía trước các ký tự ` ' " \ NUL` trong chuỗi $str
  - Thường dùng trong các câu lệnh SQL để tránh xảy ra lỗi khi lưu và lấy dữ liệu từ CSDL.
- Các hàm tương tự:
  - string **quotemeta**(string $str)
  - string **addcslashes**(string $str, string $charlist)
- Ví dụ:

```php
<?php
    $str = "I'dont know \'every thing\'";
    echo addslashes($str);
    // Output: I\'dont know \\\'every thing\\\'
?>
```

Một số hàm PHP thường dùng

string **stripslashes**(string $str)

- **Xóa bỏ** ký tự backslash (\) xuất hiện trong chuỗi $str (ngược lại với addslashes).
- Hàm tương tự:
  - string **stripcslashes**(string $str)
- Ví dụ:

```php
<?php
    $str = "I\'dont know \\\'every thing\\\'";
    echo stripslashes($str);
    // Output: I'dont know \'every thing\'
?>
```

Một số hàm PHP thường dùng

string **htmlspecialchars**(string $str [, int $flags [, string $encoding]])

- Chuyển đổi các ký tự đặc biệt **&** **'** **"** **<** **>** trong chuỗi $str thành các thực thể HTML.
- Khi đó:
  - **&** → &
  - **"** → " // khi không có **ENT_NOQUOTES**.
  - **'** → ' // khi có **ENT_QUOTES**.
  - **<** → <
  - **>** → >
- Ngược lại: **htmlspecialchars_decode** (http://php.net/manual/en/function.htmlspecialchars-decode.php)

Một số hàm PHP thường dùng

string **md5**(string $str [, bool $raw_output])

- **MD5**: Message Digest 5 là một *hàm băm mật mã*, tạo ra chuỗi mã băm có chiều dài cố định **128** bit từ chuỗi dữ liệu đầu vào $str có chiều dài tùy ý.
- Thường dùng để mã hóa mật khẩu, kiểm tra tính toàn vẹn của tập tin,...
- Ví dụ:

```php
<?php
    $str = 'Lớp DH8TH';
    echo md5($str);
    // Output: 39a03156031b6a3ecf5dc5279ab3a77c
?>
```

Một số hàm PHP thường dùng

string **sha1**(string $str [, bool $raw_output])

- Thuật toán **SHA-1** (Secure Hash Algorithm 1) tạo ra chuỗi mã băm có chiều dài cố định **160** bit từ chuỗi dữ liệu đầu vào $str có chiều dài tùy ý.
- Được sử dụng phổ biến và có công dụng như MD5, ngoài ra SHA-1 còn được sử dụng rất nhiều trong thương mại điện tử, tạo chữ ký số,...
- Ví dụ:

```php
<?php
    $str = 'Lớp DH8TH';
    echo sha1($str);
    // Output: a6fc5d0530e75a5288e4ff27b284741945677158
?>
```

Một số hàm PHP thường dùng

- Hàm ngày, giờ:
  - **date**($format [, $timestamp]) – Lấy thời gian (hiện tại) theo định dạng $format.
  - **getdate**([$timestamp]) – Trả về một mảng thông tin về thời gian hiện tại.
  - **mktime**($hour, $minute, $second, $month, $day, $year) – Trả về số giây.
  - **strtotime**($str) – Trả về số giây (Unix timestamp) từ chuỗi thời gian $str.
  - **time**(void) – Trả về số giây (Unix timestamp).
  - **date_default_timezone_set**($timezone) – Thiết lập múi giờ $timezone.

Một số hàm PHP thường dùng

- Hàm ngày, giờ:
  - string **date**(string $format [, int $timestamp])
  - Lấy thời gian (hiện tại) theo định dạng $format.

```php
<?php
    $today = date("F j, Y, g:i a");   // August 28, 2017, 12:10 am
    $today = date("d.m.y");           // 28.08.17
    $today = date("j/n/Y");           // 28/8/2017
    $today = date("dmY");             // 28082017
    $today = date("h-i-s, j-m-y");    // 12-10-43, 28-08-17
    $today = date("D M j G:i:s Y");   // Mon Aug 28 0:10:43 2017
    $today = date("H:i:s");           // 00:10:43
?>
```

  - Xem thêm các định dạng: http://php.net/manual/en/function.date.php

Một số hàm PHP thường dùng

- Hàm ngày, giờ:
  - array **getdate**([int $timestamp]) – Trả về một mảng thông tin về thời gian hiện tại.

```
Array(
    [seconds]   => 6
    [minutes]   => 18
    [hours]     => 0
    [mday]      => 28
    [wday]      => 1
    [mon]       => 8
    [year]      => 2017
    [yday]      => 239
    [weekday]   => Monday
    [month]     => August
    [0]         => 1503879486
)
<?php $now = getdate(); echo $now['year'] // Output: 2017 ?>
```

Một số hàm PHP thường dùng

- Hàm ngày, giờ:
  - int **mktime**(int $hour, int $minute, int $second, int $month, int $day, int $year)
  - Trả về số giây (Unix timestamp) ứng với các tham số đã cho.
  - Unix timestamp là thời gian được tính bằng giây kể từ 0:0:0 1/1/1970 GMT đến mốc được xác định.

```
<?php
    echo date("M-d-Y", mktime(0, 0, 0, 12, 32, 1997));
    echo date("M-d-Y", mktime(0, 0, 0, 13, 1, 1997));
    echo date("M-d-Y", mktime(0, 0, 0, 1, 1, 1998));
    echo date("M-d-Y", mktime(0, 0, 0, 1, 1, 98));
?>
```

Một số hàm PHP thường dùng

- Hàm ngày, giờ:
  - int **strtotime**(string $str)
  - Trả về số giây (Unix timestamp) từ chuỗi thời gian $str.

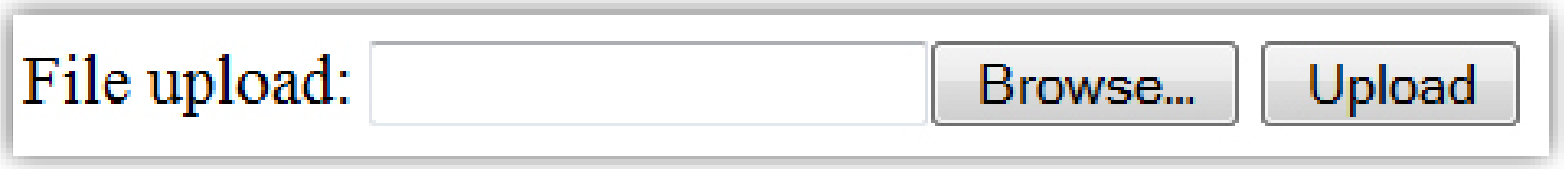
```php
<?php
    echo strtotime("now");
    echo strtotime("10 September 2016");
    echo strtotime("+1 day");
    echo strtotime("+1 week");
    echo strtotime("+1 week 2 days 4 hours 2 seconds");
    echo strtotime("next Thursday");
    echo strtotime("last Monday");
?>
```

  - Xem thêm định dạng chuỗi thời gian: http://php.net/manual/en/datetime.formats.relative.php

Một số hàm PHP thường dùng

- Form upload:

```
<form action="upload.php"  method="post" enctype="multipart/form-data">
    File upload:
    <input type="file" name="myfile" />
    <input type="submit" value="Upload" />
</form>
```



- Lưu ý:
  - Luôn sử dụng phương thức POST.
  - Luôn sử dụng enctype="multipart/form-data" trong thẻ FORM.

Upload file lên server

- Biến $_FILES[]:
  - `$_FILES['myfile']['name']`
    - Tên file đã được upload.
  - `$_FILES['myfile']['type']`
    - Kiểu MIME file đã được upload, vd: image/gif, image/jpeg,...
  - `$_FILES['myfile']['size']`
    - Kích thước tập tin đã được upload (tính bằng bytes).
  - `$_FILES['myfile']['tmp_name']`
    - Vị trí file được lưu trữ tạm trên server khi chưa upload xong.
  - `$_FILES['myfile']['error']`
    - Mã lỗi của việc upload (0 = Upload thành công).

Upload file lên server

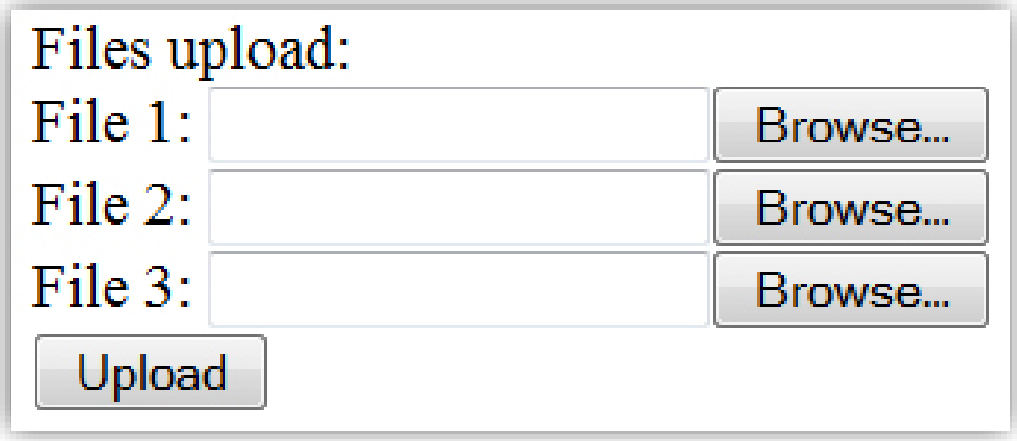
- Xử lý:

```php
<?php
    $dir = "data/"; // Upload vào thư mục data
    if($_FILES['myfile']['name'] != "") {
        $fileupload = $dir . $_FILES['myfile']['name'];
        if(move_uploaded_file($_FILES['myfile']['tmp_name'], $fileupload)) {
            echo "Upload file thành công!";
        }
        else {
            echo "Upload file không thành công!";
        }
    }
    else {
        echo "Vui lòng chọn file để upload!";
    }
?>
```

Upload file lên server

- Upload nhiều file:

```
<form action="upload.php" method="post" enctype="multipart/form-data">
    Files upload:<br />
    File 1: <input type="file" name="myfile[]" /><br />
    File 2: <input type="file" name="myfile[]" /><br />
    File 3: <input type="file" name="myfile[]" /><br />
    <input type="submit" value="Upload" />
</form>
```



Upload file lên server

- Xử lý upload nhiều file:

```php
<?php
    foreach($_FILES['myfile']['error'] as $key => $error)
    {
        if($error == 0)
        {
            $tmp_name = $_FILES['myfile']['tmp_name'][$key];
            $name = $_FILES['myfile']['name'][$key];
            move_uploaded_file($tmp_name, "data/$name");
        }
    }
?>
```

Nội dung chính của slide này

- Giới thiệu
- Làm sao “chạy” được PHP?
- Cơ bản về PHP
- Biến “Superglobals”
- Form
- Mảng
- Hàm
- Một số hàm PHP thường dùng
- **Session**
- Hướng đối tượng trong PHP

Session

- **Session** là một phiên làm việc kể từ khi người dùng truy cập vào website cho đến khi **đóng trình duyệt** hay **abandon** (kết thúc) phiên.
- PHP đưa ra biến toàn cục `$_SESSION[]` để lưu trữ thông tin Session, và biến này có thể truy cập từ mọi trang PHP trong phiên.
- Khai báo và thiết lập Session:

```php
<?php
    session_start(); // Phải đặt trước thẻ <html>
    $_SESSION['name']="Nguyễn Hoàng Tùng";
    header("Location: index.php");
?>
```

Session

- Xóa bỏ Session:
  - `unset($_SESSION['name']);` hoặc
  - `session_destroy();` // Hủy bỏ toàn bộ biến $_SESSION[]
- Đây là các câu lệnh thường dùng khi "*đăng xuất*" hệ thống, tuy nhiên khi dùng hàm **session_destroy()** phải lưu ý, vì có xóa những biến Session khác của hệ thống hay không?

Session

- Tóm tắt:
  - Khai báo
    - `session_start(); $_SESSION['tên'] = giá_trị;`
  - Sử dụng
    - `session_start(); $biến = $_SESSION['tên'];`
  - Xóa bỏ Session đã khai báo
    - `unset($_SESSION['tên']);`
  - Hủy bỏ toàn bộ biến $_SESSION[]
    - `session_destroy();`
  - Session có rất nhiều ứng dụng, ví dụ như lưu trữ thông tin người dùng khi đăng nhập thành công, lưu giỏ hàng trong e-Commerce.

Nội dung chính của slide này

- Giới thiệu
- Làm sao “chạy” được PHP?
- Cơ bản về PHP
- Biến “Superglobals”
- Form
- Mảng
- Hàm
- Một số hàm PHP thường dùng
- Session
- **Hướng đối tượng trong PHP**

Hướng đối tượng trong PHP

- Giới thiệu.
- Khai báo lớp.
- Truy cập các thành phần của lớp bằng **$this**.
- Hàm tạo (**__construct**).
- Hàm hủy (**__destruct**).
- Thành phần **static**.
- Kế thừa (**extends**).

Hướng đối tượng trong PHP

- Giới thiệu:
  - OOP chỉ có trong phiên bản từ PHP 5 về sau.
  - Đặc điểm OOP trong PHP: Không cho phép đa kế thừa!

Hướng đối tượng trong PHP

- Khai báo lớp:

```
class tên_lớp
{
    // Khai báo thuộc tính
    // Khai báo phương thức
    // Sử dụng private hay public và protected để xác định phạm vi
}
```

- Tất cả các thành phần của một lớp đều phải được khai báo tính "đóng gói" với các từ khoá tương ứng là **public**, **protected** hoặc **private**.

Hướng đối tượng trong PHP

- **Không** khai báo lớp:
  - Trong nhiều file,
  - Trong nhiều khối lệnh.
- Sử dụng:
  - Tạo biến đối tượng:
    - `$tên_object = new tên_class([giá trị khởi tạo]);`
    - `$sv = new SinhVien("Nguyễn Tùng");`
  - Sử dụng thuộc tính, phương thức:
    - `$tên_object->tên_thuộc_tính;`
    - `$tên_object->tên_phương_thức([tham số]);`

Hướng đối tượng trong PHP

- Ví dụ đơn giản:

```php
<?php
    class SinhVien {
        public $mSSV = "";
        public $hoTen = "";

        public function __construct($mS, $hTen) { // Hàm dựng
            $this->mSSV = $mS;
            $this->hoTen = $hTen;
        }
    }

    $sv = new SinhVien("DTH051187", "Nguyễn Tùng");
    echo "Mã số SV: " . $sv->mSSV . ", Họ tên: " . $sv->hoTen;
?>
```

Hướng đối tượng trong PHP

- Khi sử dụng các biến thành phần, **bắt buộc** phải sử dụng từ khóa $this.

```php
<?php
    class SinhVien {
        public $mSSV = "";
        public $hoTen = "";

        public function __construct($mS, $hTen) { //Hàm dựng
            $this->mSSV = $mS;
            $this->hoTen = $hTen;

            $this->$mSSV = $mS;
            $this->$hoTen = $hTen;
        }
    }
?>
```

Đúng!

Sai!

Hướng đối tượng trong PHP

- Nạp chồng hàm (Overloading):
  - **Không** cho phép nạp chồng hàm.
  - Để giải quyết vấn đề, ta sử dụng hàm có tham số mặc định.

```php
<?php
    class SinhVien {
        private $mSSV = "";
        private $hoTen = "";

        public function __construct($mS = "", $hTen = "") {
            $this->mSSV = $mS;
            $this->hoTen = $hTen;
        }
    }
?>
```

Hướng đối tượng trong PHP

- Hàm tạo (Constructor):
  - Hàm tạo cho phép khởi tạo giá trị cho thuộc tính của đối tượng.
  - Tên hàm tạo: **__construct()**
  - PHP sẽ tự động gọi hàm tạo khi tạo một đối tượng từ một lớp.
- Ví dụ:

```
class Fruit {
    public $name;
    public $color;

    public function __construct($name) {
        $this->name = $name;
    }
}
```

Hướng đối tượng trong PHP

- Hàm hủy (Destructor):
  - **__destruct()**
  - Thường được dùng để dọn dẹp, đóng kết nối tới CSDL.

```php
class Fruit {
    public $name;
    public $color;

    public function __construct($name, $color) {
        $this->name = $name;
        $this->color = $color;
    }
    public function __destruct() {
        echo "Thực thi hàm hủy.";
    }
}
```

Hướng đối tượng trong PHP

- Đối tượng static:
  - Lưu ý, **không** áp dụng **static** đối với hàm tạo.

```php
<?php
    class SinhVien {
        private static $mSSV = "";
        private $hoTen = "";

        public function __construct($mS = "", $hTen = "") {
            self::$mSSV = $mS;
            $this->hoTen = $hTen;
        }
    }
?>
```

Hướng đối tượng trong PHP

- Đối tượng static:
  - Lưu ý, **không** sử dụng **new** để khai báo đối tượng.

```php
<?php
    class SinhVien {
        private static $mSSV = "";
        private static $hoTen = "";

        public static function HienThi() {
            echo "MSSV: " . self::$mSSV;
            echo "Họ tên: " . self::$hoTen;
        }
    }

    SinhVien::HienThi();
?>
```

Hướng đối tượng trong PHP

- Kế thừa:
  - Dùng từ khóa **extends** để thể hiện kế thừa.

```php
<?php
    class SinhVienTH extends SinhVien {
        private $soTinChi = 0;
        public function __construct($mS, $hTen, $sTC) {
            parent::__construct($mS, $hTen);
            $this->soTinChi = $sTC;
        }
        public function HienThi() {
            parent::HienThi();
            echo "Số tín chỉ: " . $this->soTinChi;
        }
    }
?>
```

Giải đáp thắc mắc